Supplemental information

How do RNNs encode mixtures? One network yields distinct geometries

Esther Poniatowski · Claire Sergent • INCC, CNRS UMR 8002, Université Paris Cité

DERIVATIONS, ASSUMPTIONS & QUANTITATIVE PREDICTIONS

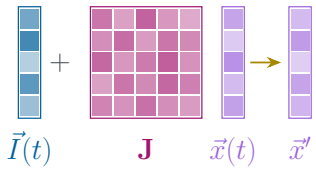
1 Model

A few biologically plausible assumptions reduce an established neuron-level model to a few coordinates that capture output geometry:

(1) Standard rate model N neurons with recurrent connectivity and external input

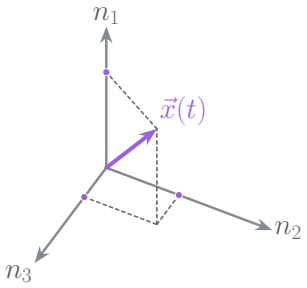
Each neuron i carries an activation x_i that relaxes with time constant τ, is driven by the external input I_i^{ext}, and receives the firing rates $\phi(x_j)$ of the other neurons through the recurrent weights J_{ij}, with activation function ϕ:

$$\tau \frac{\mathrm{d}x_i}{\mathrm{d}t} = -x_i + \sum_{j=1}^{N} J_{ij}\, \phi(x_j) + I_i^{\text{ext}}$$



implementation

$\Rightarrow$ The network's activity state $\vec{x}$ has N coordinates, one per neuron.



activity space

(2) Structured connectivity R interaction patterns and S input patterns

The connectivity decomposes into a sum of R matrices of **rank one**, with $R \ll N$ (Mastrogiuseppe & Ostojic, 2018). Each term pairs two population patterns:

$$\mathbf{J} = \tfrac{1}{N} \sum_{r=1}^{R} \vec{m}^{(r)} \vec{n}^{(r)\top}$$

Selection filter $\vec{n}^{(r)}$

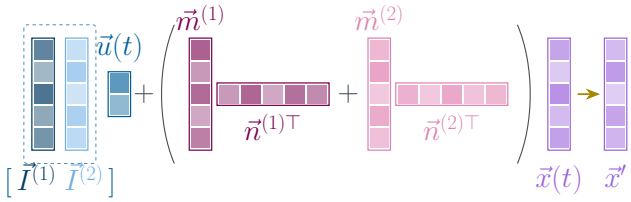
- It is a pattern the network is tuned to, imprinted or emerging through learning.
- It reads the population activity out, returning how strongly that pattern is present.

Response pattern $\vec{m}^{(r)}$

- It gathers the strengths with which each neuron answers the filter $\vec{n}^{(r)}$, across the whole population.
- It gives the direction along which the network feeds that signal back.

The external input is likewise a combination of S fixed patterns $\vec{I}^{(s)}$ with amplitudes u_s:

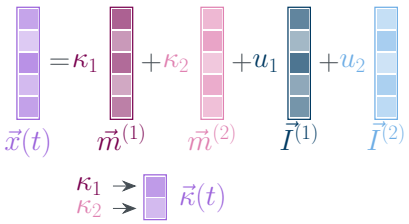
$$\vec{I}^{\text{ext}} = \sum_{s=1}^{S} \vec{I}^{(s)} u_s$$



implementation

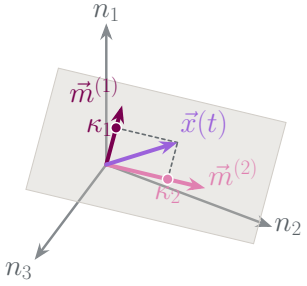
Substituting the decomposition into the stationary state $\vec{x} = \mathbf{J}\phi(\vec{x}) + \vec{I}^{\text{ext}}$ constrains the activity: every recurrent contribution is a multiple of some $\vec{m}^{(r)}$, so the network's state $\vec{x}$ is confined to the subspace spanned by the R response patterns and the S input patterns:

$$\vec{x} = \sum_{r=1}^{R} \kappa_r \vec{m}^{(r)} + \sum_{s=1}^{S} u_s \vec{I}^{(s)}, \qquad \kappa_r = \tfrac{1}{N} \vec{n}^{(r)\top} \phi(\vec{x}), \qquad \vec{\kappa} = (\kappa_1, \ldots, \kappa_R)$$



reduced state

$\Rightarrow$ The reduced state $\vec{\kappa}$ collects the activity's coordinates *along the recurrent response patterns* $\vec{m}^{(r)}$: $R \ll N$ coordinates in place of the original N.



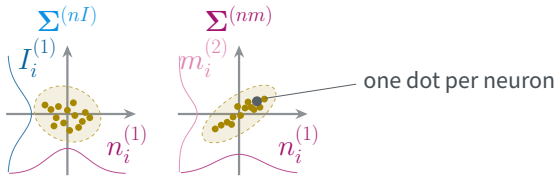
activity space

(3) Gaussian weight distribution pattern entries jointly Gaussian across neurons

Each neuron's entries in all the patterns are drawn *together* from one zero-mean Gaussian, independently from neuron to neuron (Beiran et al., 2021; Dubreuil et al., 2022):

$$\left(m_i^{(r)},\, n_i^{(r)},\, I_i^{(s)}\right)_{r\le R,\, s\le S} \sim \mathcal{N}\left(\vec{0},\, \mathbf{\Sigma}\right), \qquad i = 1, \ldots, N$$

A network is therefore specified by the **covariances between pattern families** — the blocks $\mathbf{\Sigma}^{(nI)}$, $\mathbf{\Sigma}^{(nm)}$, $\mathbf{\Sigma}^{(II)}$ and $\mathbf{\Sigma}^{(mm)}$ of $\mathbf{\Sigma}$ — and not by any individual weight.



pattern statistics

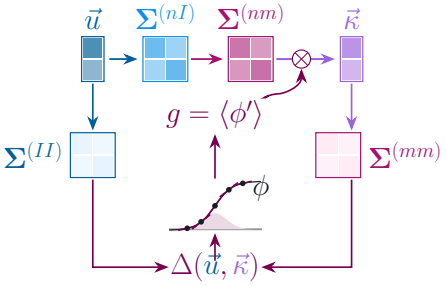
Because the pattern distributions are *centred*, each neuron's total input is a zero-mean Gaussian across the population, so its law is settled by one number — the variance of that input across neurons:

$$\Delta = \Delta_0 + \vec{u}^{\top} \mathbf{\Sigma}^{(II)} \vec{u} + \vec{\kappa}^{\top} \mathbf{\Sigma}^{(mm)} \vec{\kappa}$$

A neuron's response to a *small* change of its input is set by the slope of the activation function ϕ at its current activation — its **sensitivity**. Activations differ from neuron to neuron, so the population is characterised by the *average* slope; and since the activations are Gaussian with variance Δ, that average depends on the variance alone. That average slope is the **gain** g:

$$g(\Delta) = \mathbb{E}_{Z\sim\mathcal{N}(0,1)}\left[\phi'(\sqrt{\Delta}\, Z)\right]$$

$\Rightarrow$ Only now does the model collapse: for $N \to \infty$ the activity is determined by those covariances and the single gain alone — an effective **network**.



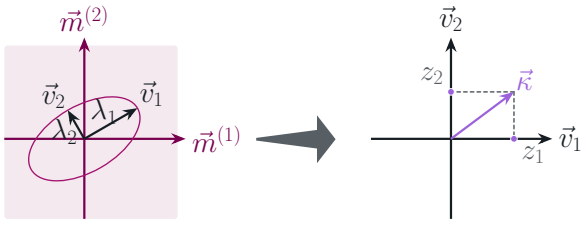
gain loop

(4) Change of basis eigenvectors of the recurrent covariance $\mathbf{\Sigma}^{(nm)}$

At this point, the activity can be expressed in the basis of the response patterns. In that basis, however, the stationary equation stays **coupled across coordinates**: the value of each coordinate depends on the values of all the others, so no single quantity characterises a given direction on its own.

Diagonalising $\mathbf{\Sigma}^{(nm)}$ removes the coupling. In its eigenbasis the coordinates separate, so the recurrent contribution reduces to **one scalar factor per direction**. The basis is fixed by the connectivity alone, so a mixture and its components are always compared in the **same frame**, whose axes are the eigendirections $\vec{v}_k$ with eigenvalues λ_k.

$\Rightarrow$ The output has R coordinates z_k, one per eigendirection $\vec{v}_k$ — the **intrinsic basis**.



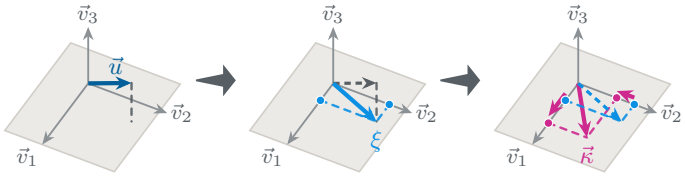
recurrent plane intrinsic coordinates

2 Two-transformation input–output map

Once the network is reduced, its **input–output map** is a single expression: it takes the input amplitudes to the reduced state, in two successive transformations.

$$\vec{\kappa} = \underbrace{g\left(\mathbf{I} - g\,\mathbf{\Sigma}^{(nm)}\right)^{-1}}_{\text{reweighting}}\ \underbrace{\mathbf{\Sigma}^{(nI)}}_{\text{projection}}\ \vec{u}$$

$\vec{\kappa}$: reduced state, $\vec{u}$: input amplitudes, $\mathbf{\Sigma}^{(nI)}$: covariance between the selection filters and the input patterns, $\mathbf{\Sigma}^{(nm)}$: recurrent covariance, g: gain, $\mathbf{I}$: identity.



an input projected reweighted

Projection — linear

- It acts on the input amplitudes $\vec{u}$.
- It returns one coordinate per recurrent pattern: the input resolved onto the selection filters, in the basis of the response patterns $\vec{m}^{(r)}$.
- It is linear in the input, and contributes none of the map's non-linearity.

Reweighting — non-linear

- It acts on the coordinates the projection returns.
- It returns the reduced state $\vec{\kappa}$, each coordinate re-injected through the recurrent covariance $\mathbf{\Sigma}^{(nm)}$.
- It is linear once the gain is held fixed, so the map's non-linearity enters entirely through the gain's self-consistency.

The source of that non-linearity is the gain's **self-consistency**. The gain is set by the variance; the variance contains the state; and the state is returned by the map itself. The three are therefore solved together, and the gain moves with the input. A map that is linear at fixed gain is in this way non-linear in the input.

3 Coordinate factorization

Written in the **eigenvector basis** of the recurrent covariance, the map separates. The matrix inverse of the previous point reduces to one scalar per direction, so each coordinate carries the same two transformations as a product of two numbers:

$$z_k = \underbrace{\frac{g}{1-g\lambda_k}}_{A_k(g)} \underbrace{\vec{v}_k^{\top}\boldsymbol{\Sigma}^{(nI)}\vec{u}}_{\xi_k}$$

reweighting: $\frac{g}{1-g\lambda_k}$ = $A_k(g)$; projection: $\vec{v}_k^{\top}\boldsymbol{\Sigma}^{(nI)}\vec{u}$ = ξ_k

z_k: coordinate on the kth eigendirection, $\vec{v}_k$: eigenvector, λ_k: eigenvalue; g: gain, A_k: reweighting factor, ξ_k: input projection.

The two factors play **asymmetric** roles, and they cannot be varied independently of one another: the recurrent factor acts only on directions that the input already drives.

Projection ξ_k — the gate

- It measures the drive the input delivers to direction k, once resolved through the selection filters.
- It determines whether a direction is open at all, and so whether the output can leave the components plane.

Reweighting A_k — the selector

- It measures the recurrent gain of that direction alone, set by its eigenvalue λ_k and the global gain g.
- It governs how the open directions are weighted against one another, and so which geometry is realized.

The conditions below therefore compare the reweighting factors *across* directions.

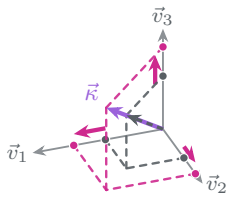
4 Condition for pure scaling

When does a mixture's output stay aligned with the linear sum of the component outputs? This is the *unbiased, in-plane* case. Throughout, each component alone is taken to elicit the same scalar gain g_c, so a mixture differs from its components only through its own gain g_{mix}.

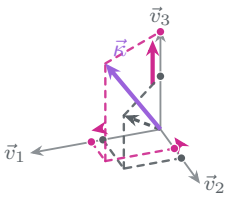
Alignment holds exactly when the reweighting-factor ratio $A_k(g_{\text{mix}})/A_k(g_c)$ (3) is **uniform** across the active directions — a **flat** reweighting profile. A uniform ratio multiplies every active coordinate by the same scalar, acting on the projected mixture **homogeneously** rather than direction by direction:

$$z_k = A\, z_k^{\text{sum}} \quad \text{for every active } k \qquad \Longleftrightarrow \qquad \vec{\kappa}_{\text{mix}} = A\,\vec{\kappa}_{\text{sum}}$$

The output is then the linear sum **rescaled**. Its geometry is untouched and only the magnitude changes: this is the unbiased, in-plane case. In contrast, a non-uniform ratio stretches the active directions by unequal amounts and so **rotates** the output away from the linear sum. The bias and emergence properties measure that rotation.



flat reweighting: every coordinate scales alike, the direction is preserved



uneven reweighting: each coordinate scales differently, the direction turns

Writing $A_k(g) = g/(1 - g\lambda_k)$ turns the condition on the ratio into a condition on the **spectrum**, because the ratio depends on the direction only through its eigenvalue:

$$\frac{A_k(g_{\text{mix}})}{A_k(g_c)} = \frac{g_{\text{mix}}}{g_c} \cdot \frac{1 - g_c\lambda_k}{1 - g_{\text{mix}}\lambda_k}$$

The leading factor is common to every direction, so uniformity rests entirely on the second one. It is constant across the active directions in exactly two circumstances, and they are the two regimes the mechanism can occupy:

Distinct active eigenvalues

- The eigenvalue factor varies from direction to direction, since each carries a different λ_k.
- Uniformity therefore demands that the two gains coincide, $g_{\text{mix}} = g_c$.
- A mixture changes the variance, and hence the gain, so that equality fails.
- **Every geometry becomes reachable** as the input varies.

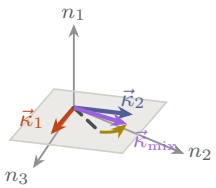
Equal active eigenvalues

- The eigenvalue factor takes the same value in every direction, since the λ_k coincide.
- Uniformity demands no further condition, the ratio being already common to all directions.
- That holds for any gain, whatever the mixture does to the variance.
- The output stays the rescaled linear sum, so **only the unbiased, in-plane geometry** is available.

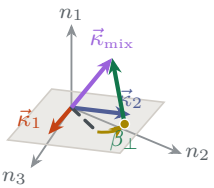
5 Condition for in-plane geometry

When can a mixture's output stay in the components plane? Written in the frame of the two component outputs, staying in that plane means the orthogonal coefficient vanishes, so only β_1, β_2 remain free:

$$\vec{\kappa}_{\text{mix}} = \underbrace{\beta_1}_{?}\,\vec{\kappa}_1 + \underbrace{\beta_2}_{?}\,\vec{\kappa}_2 + \underbrace{\beta_\perp}_{0}\,\vec{\eta}_\perp$$



the output stays
in the components plane



the output leaves
the components plane

Both sides are read in the intrinsic basis, one coordinate at a time. On the left, the mixture's own coordinate is its projection reweighted by its own gain, and the projection is *linear* in the input amplitudes, so the mixture's projection is the combination of the components':

$$z_k^{\text{mix}} = A_k(g_{\text{mix}})\,\xi_k^{\text{mix}} \qquad \text{with} \qquad \xi_k^{\text{mix}} = \alpha_1\xi_k^{(1)} + \alpha_2\xi_k^{(2)}$$

On the right, each component alone is reweighted by the common component gain g_c, so its own coordinate carries the same factor:

$$z_k^{(i)} = A_k(g_c)\,\xi_k^{(i)}$$

Staying in the plane means the output is the pair β_1, β_2 applied to the component outputs. Coordinate by coordinate that reads:

$$z_k^{\text{mix}} = \beta_1 z_k^{(1)} + \beta_2 z_k^{(2)} = A_k(g_c)\big(\beta_1 \xi_k^{(1)} + \beta_2 \xi_k^{(2)}\big)$$

Equating the two expressions for z_k^{mix} and dividing by $A_k(g_c)$, which is non-zero on an active direction, leaves **one scalar equation per active eigendirection**, while the unknowns remain the **two** coefficients β_1, β_2:

$$\tfrac{A_k(g_{\text{mix}})}{A_k(g_c)}\big(\alpha_1 \xi_k^{(1)} + \alpha_2 \xi_k^{(2)}\big) = \beta_1 \xi_k^{(1)} + \beta_2 \xi_k^{(2)} \qquad \text{for every active } k$$

Whether the output can stay in the plane therefore reduces to **counting equations against unknowns**, and the count is fixed by the number of active directions:

Two active directions

- Two equations constrain two unknowns.
- The system is square, so it admits a solution for generic values.
- The pair β_1, β_2 reproduces the output on every active direction.
- The output **stays in the components plane**.

Three or more active directions

- The equations outnumber the unknowns.
- An overdetermined system has no solution for generic values.
- No pair β_1, β_2 reproduces the output on all of them.
- The output acquires an orthogonal part and **leaves the components plane**.

Both verdicts are generic. The exceptions are the fine-tuned inputs or connectivities that make an overdetermined system consistent. An emergent geometry is therefore the **generic** outcome once three directions are active, rather than one possibility among equals.

6 Input variance vs. similarity

The conditions of points 4 and 5 are both read off the comparison between a mixture's gain and its components', and both presuppose that the two differ. That difference has not yet been accounted for. *Why does a mixture drive the network at a different gain from its components?* The gain depends on the input through one quantity alone, the input's share of the total variance (1), so the answer lies in the effect of mixing on that variance.

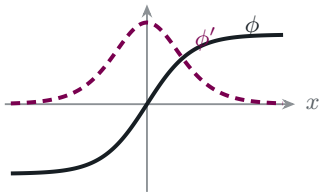
$$\Delta^{(I)}(\vec{u}) = \vec{u}^{\top} \mathbf{\Sigma}^{(II)} \vec{u}, \qquad \vec{u}_{\text{mix}} = \alpha_1 \vec{u}_1 + \alpha_2 \vec{u}_2, \qquad \alpha_1 + \alpha_2 = 1$$

Expanding the quadratic form at $\vec{u}_{\text{mix}}$ and collecting the terms against the weighted average of the components' own variances leaves an exact identity:

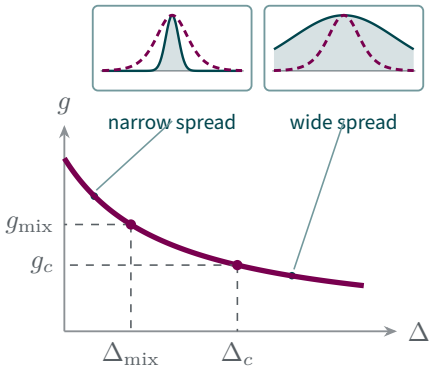
$$\Delta_{\text{mix}}^{(I)} = \underbrace{\alpha_1 \Delta_1^{(I)} + \alpha_2 \Delta_2^{(I)}}_{\bar{\Delta}_{\text{comp}}^{(I)} \text{ (weighted average)}} - \underbrace{\alpha_1 \alpha_2 \,(\vec{u}_1 - \vec{u}_2)^{\top} \mathbf{\Sigma}^{(II)} (\vec{u}_1 - \vec{u}_2)}_{\text{squared distance between the input patterns}}$$

The subtracted term is a **squared distance** between the two input patterns, measured in the metric $\mathbf{\Sigma}^{(II)}$. Being a square it is never negative, so **mixing cannot raise the input variance above the components' average**. The term vanishes only when the two patterns coincide, and grows as they separate: in this sense the drop measures *dissimilarity*. For fixed patterns it is largest at balanced mixing, where the product $\alpha_1 \alpha_2$ peaks.

The gain depends on the input only through that variance, and the activation function saturates. A wider spread of activations brings a larger fraction of neurons into the saturating range, where the slope of the activation function is small, so averaging those slopes returns a smaller gain: the gain **decreases** with the variance. A mixture therefore sits at a lower variance and a higher gain than its components, on the same curve.



the activation function
saturates, so its slope collapses



the gain falls as the variance grows

$\Rightarrow$ A mixture generically runs at a gain g_{mix} different from the components' common g_c. Points **4** and **5** convert that inequality into geometry. The difference between the two gains widens as the components become more dissimilar, and the departure from the linear sum widens with it.

7 Divisive normalization vs. network model

The standard account of neural responses to mixtures is **divisive normalization**, in which each component's weight is normalized by the summed activity of all components. The two accounts are set against each other row by row.

Divisive normalization

Carandini & Heeger 2012

- **Descriptive** — fits **phenomenology**, does not explain.
- Captures outputs only in the components plane (unbiased and biased) — **no emergence**.
- Bias **keeps growing** with input unbalance.

Recurrent network model

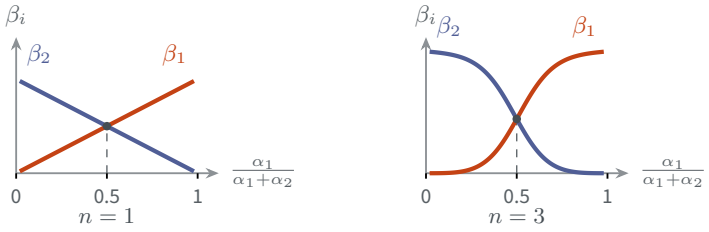
this work

- **Mechanistic** — derives outputs **from network connectivity**.
- Captures **every geometry**, emergence included.
- Bias **saturates** with input unbalance.

Why can divisive normalization not produce emergence? (row 2) The rule assigns one weight to each component, and produces no further coefficient:

$$\beta_i^{\mathrm{DN}} = \frac{\alpha_i^n}{\sigma^n + \alpha_1^n + \alpha_2^n}$$

The rule assigns the two weights together, and the exponent governs how sharply one takes over from the other as the input unbalances:



⇒ The output is a weighted sum of the components *by construction*, so it cannot leave their plane: the rule provides no orthogonal coefficient $\beta_\perp$. Emergence lies outside the model's reach, and no choice of the exponent n or the constant σ brings it within.

Why do the two biases behave differently? (row 3) The bias is measured by the relation between the **output weights** β_1, β_2 and the **input weights** α_1, α_2: how far the output ratio departs from the input ratio. Both accounts give that relation the same form, a power of the input ratio, and part on the exponent:

$$\frac{\beta_1^{\mathrm{DN}}}{\beta_2^{\mathrm{DN}}} = \left(\frac{\alpha_1}{\alpha_2}\right)^n \qquad n_{\mathrm{net}} = \frac{1 - g_c\lambda_2}{1 - g_{\mathrm{mix}}\lambda_2}\,\frac{1 - g_{\mathrm{mix}}\lambda_1}{1 - g_c\lambda_1}$$

A **fixed** exponent

- The exponent is a constant of the model, fitted to the data.
- As the input unbalances it stays fixed, so the output ratio remains the same power of the input ratio.
- The bias therefore **grows without bound**.

An **input-dependent** exponent

- The equivalent exponent is set by the gains and the connectivity eigenvalues, not fitted.
- As the input unbalances it falls, because the mixture's gain moves with the input (6).
- The bias therefore **saturates**.

The two laws agree at balance and part as the input becomes unbalanced. At the balance point they predict the same output ratio, so no measurement taken there separates them.

